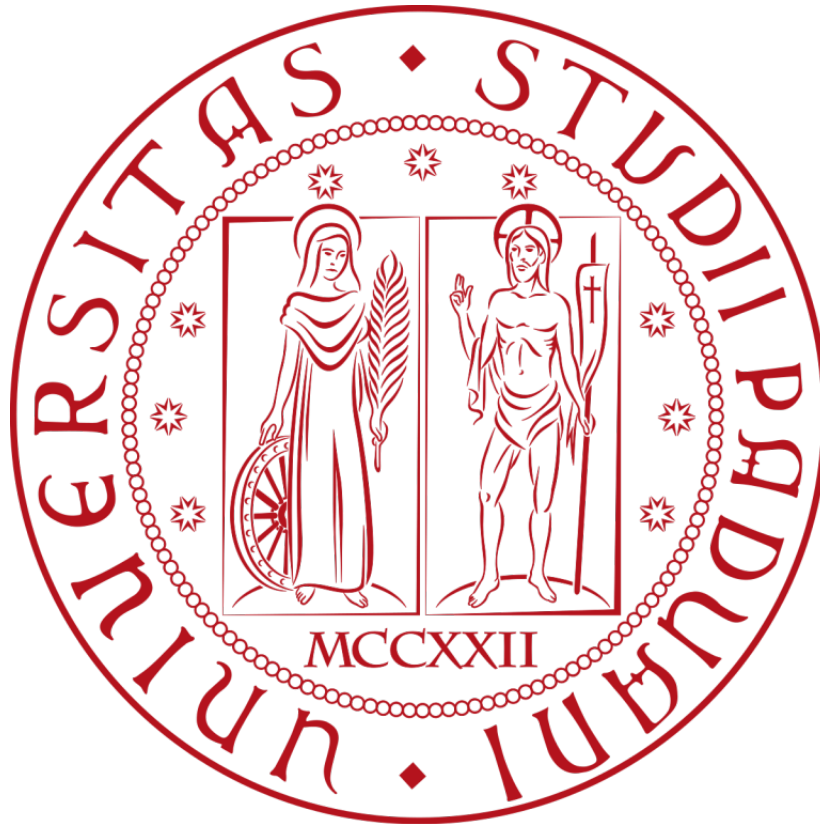


Implementation of Frank-Wolfe Methods for Recommender Systems

Diego A. Brule Galleguillos
Diana C. Andrade Damian
Evgeni Markin
Marina Lima Braga

July 2025



1 Introduction

Recommender systems are all around us in our technologically dense modern life: From Spotify’s personalised playlists, TikTok’s for you page, or Netflix’s recommended movies and series for each user, each represents the same underlying problem, a sparse matrix completion problem. While representations may be slightly different depending on the application context, the question remains the same: ”If a user has consumed certain pieces of media, what else are they likely to consume?” The matrix completion problem answers this question by completing missing entries to a seen matrix representing users and consumed media. Guided by the constraint of keeping the matrix low-rank, the seen matrix’s missing values in each row are filled in so that they are similar to other rows, which results in rows with similar seen entries having their missing values filled similarly. This means that a user who watched four movies in common with another user, will be recommended a fifth movie that the other user watched. Or in the wording we often see while using these platforms: ”Other users with a similar taste also liked:”.

2 Matrix Completion Problem

The matrix completion problem that we are attempting to solve here can be stated as the following: Find a low rank matrix $X \in \mathbb{R}^{n_1 \times n_2}$ from a set of entries taken from an observed sparse matrix R of the same dimensions. This has the following mathematical formulation:

$$\min_{X \in \mathbb{R}^{n_1 \times n_2}} f(X) := \sum_{(i,j) \in J} (X_{ij} - R_{ij})^2 \quad \text{s.t.} \quad \text{rank}(X) \leq \delta \quad (1)$$

This can be seen as a loss function that we can minimise by finding the best estimated matrix X . However, the hard constraint on the rank of the matrix means this loss function is not convex, so it would be hard to solve with popular optimisation algorithms. So instead, we modify it and apply a constraint on the nuclear norm of the estimated matrix. While this is not as direct as putting a constraint on the rank itself, the nuclear norm is related to the rank, so constraining it would also constrain the rank. Since the nuclear norm is convex, this is now suitable for optimisation algorithms. This formulation is as follows:

$$\min_{X \in \mathbb{R}^{n_1 \times n_2}} \sum_{(i,j) \in J} (X_{ij} - R_{ij})^2 \quad \text{s.t.} \quad \|X\|_* \leq \delta \quad (2)$$

Having this clear formulation, the next question is, what algorithm should we implement to solve this in practice?

3 Choice of Algorithms

For solving our problem, we needed to select algorithms that could do so efficiently in high-dimensional space, since the input in the context of recommender systems are usually high-dimensional matrices, and also we needed an algorithm that would be good at computing low-rank solutions. With this in mind, we used the following algorithms, that respect those constraints, and additionally, allow for computationally cheap linear optimisation.

3.1 Franke-Wolfe Algorithm:

The Frank-Wolfe Algorithm is a clever alternative to popular gradient descent methods. Like GD methods, the FW algorithm computes a best direction of descent relative to the gradient, and takes a step in that direction. The difference lies in how it computes this step. In GD, the algorithm minimises the objective gradient itself, which may lead to a step being taken towards an estimate that is not in the problem space, which means the algorithm then has to project the estimate back to the problem space. The FW algorithm instead, computes a linear approximation of the gradient at each step, and minimises that to find a new step direction (atom). Because of this, every step taken at each iteration is simply a convex combination of previous atoms and the current atom, which in convex analysis is shown to always stay within the same convex hull, so the new estimate will always be inside the problem space and does not need projection.

Algorithm 1 Frank-Wolfe Algorithm for Matrix completion

- 1: **Input:** Observed Matrix $R \in \mathbb{R}^{n_1 \times n_2}$, index set Ω , Convex hull $\mathcal{C} = \{X \in \mathbb{R}^{n_1 \times n_2} : \|X\|_* \leq \tau\}$, objective f , initial matrix $X \in \mathcal{C}$, iterations K
 - 2: **for** $k = 0$ to K **do**
 - 3: Compute gradient: $G^{(k)} = P_{\Omega}(X^{(k)} - R)$
 - 4: Compute $S_k := \arg \min_{S: \|S\|_* \leq \tau} \langle S, G^{(k)} \rangle$
 - 5: Set step size $\gamma_k := \frac{2}{k+2}$ (Or use line search)
 - 6: Update $x^{(k+1)} := (1 - \gamma_k)X^{(k)} + \gamma_k S_k$
 - 7: **end for**
 - 8: **Return:** $X^{(K)}$
-

3.2 Pairwise Franke-Wolfe Algorithm

This FW variant has the same core ideas as the vanilla FW of using linear approximations to find atoms to build towards the final estimate, but it has the difference that it also computes an away atom, so that weight can be shifted away from an atom that is contributing poorly to the solution and a better step direction is chosen.

Algorithm 2 Pairwise Frank-Wolfe for Low-Rank Matrix Completion

- 1: **Input:** Observed Matrix $R \in \mathbb{R}^{m \times n}$, index set Ω , Convex hull $\mathcal{C} = \{X \in \mathbb{R}^{n_1 \times n_2} : \|X\|_* \leq \tau\}$, number of iterations K
 - 2: Initialize $X^{(0)} = 0$, active set $\mathcal{S}^{(0)} = \emptyset$, weights $\alpha = \emptyset$
 - 3: **for** $t = 0$ to $K - 1$ **do**
 - 4: Compute gradient: $G^{(k)} = P_\Omega(X^{(k)} - R)$
 - 5: Compute $S_{FW} := \arg \min_{S: \|S\|_* \leq \tau} \langle S, G^{(k)} \rangle$
 - 6: Compute $S_{Away} := S_{Away} = \arg \max_{S \in \mathcal{S}^{(k)}} \langle S, G^{(k)} \rangle$
 - 7: Compute Direction: $d_k = S_{FW} - S_{Away}$
 - 8: Maximum step size: $\gamma_{\max} = \alpha_{S_{Away}}$
 - 9: Line search for step size: $\gamma_k \in [0, \gamma_{\max}]$
 - 10: Update solution: $X^{(k+1)} = X^{(k)} + \gamma_k d_k$
 - 11: Update weights:
 - $\alpha_{S_{FW}} += \gamma_k$
 - $\alpha_{S_{Away}} -= \gamma_k$
 - Remove S_{Away} if $\alpha_{S_{Away}} = 0$
 - 12: Update active set $\mathcal{S}^{(k+1)}$ accordingly
 - 13: **end for**
 - 14: **return** Estimated matrix $X^{(K)}$
-

3.3 Projected Gradient Method

The Projected Gradient method is not a FW algorithm variant, however it is useful to implement as we can use it to compare performance vs FW. Projected Gradient method is more like the classical GD, where there is no linear approximation to the objective function, so a step in the direction of steepest descent may result in an estimate that is outside the problem space, in our case the convex hull. Because of this, the algorithm must also have a projection step, where if the iteration estimate is outside the hull, it projects back into it.

Algorithm 3 Projected Gradient Descent for Matrix Completion

- 1: **Input:** Observed matrix $R \in \mathbb{R}^{m \times n}$, index set Ω , step size η , nuclear norm bound τ , iterations K
 - 2: **Initialize:** $X^{(0)} = 0$
 - 3: **for** $k = 0$ to $K - 1$ **do**
 - 4: Compute gradient: $G^{(k)} = P_\Omega(X^{(k)} - R)$
 - 5: Take gradient step: $Y^{(k)} = X^{(k)} - \eta G^{(k)}$
 - 6: Compute SVD: $Y^{(k)} = U \Sigma V^\top$
 - 7: Project singular values: $\tilde{\Sigma} = \text{Proj}_{\|\cdot\|_1 \leq \tau}(\Sigma)$
 - 8: Reconstruct: $X^{(k+1)} = U \tilde{\Sigma} V^\top$
 - 9: **end for**
 - 10: **Return:** $X^{(K)}$
-

3.4 Additional comments on implementation

We implemented our algorithms using Python, and use several libraries to make our code efficient. The codes follow the pseudocodes described in this report loyally, but we will explain non-trivial operations in this subsection:

1. The gradient calculation at each iteration is done with a mask, where only the entries that were populated in the original set of seen entries contribute. Entries that were originally empty but have estimates in intermediate iterations do not contribute to the loss.
2. The calculation of the new atom at each iteration of the FW and Pairwise FW is done through singular value decomposition. By calculating the top singular vectors and values of the gradient matrix we automatically maximise the Oracle of our linear approximation to the objective function, which we can then scale accordingly using our nuclear norm constraint, and multiplication by -1 to find the new atom in a minimising direction.
3. The calculation of the away atom in the pairwise FW is done by maximising the same oracle, but with respect to the active set of atoms. For this we track the active set of atoms and then calculate the inner product of these atoms with a current gradient, and pick the atom with the highest result as the away atom.
4. Line search is performed in our algorithms to help accelerate convergence. It does so by taking the new atom, and scanning towards that direction in a pre-defined range to find the step size that would minimise the objective function approximation the most.
5. In the Projected Gradient Method, the estimate at each iteration is projected back onto the nuclear-norm ball by using the diagonal matrix Σ obtained by single-value decomposition of the intermediate estimate, and piecewise applies the operation $\max(\sigma - \theta, 0)$ which effectively projects the estimate back onto the nuclear norm ball. Theta is a parameter which is found by applying algorithm 4 described on this report.

4 Numerical Experiments

Once our algorithms were polished up and in line with the pseudocodes described in this report, we started to run numerical experiments on different datasets:

1. Lens Movies: A dataset where rows are users and columns are movies. Entries correspond to a user's rating of that movie. We used a short version of this dataset with the top 300 active users (Only 43% of entries missing) so that we could speed up the process of optimising parameters for bigger datasets.

Algorithm 4 Find threshold θ for Nuclear norm projection

Require: Vector $s \in \mathbb{R}^n$, radius $\tau > 0$

Ensure: Threshold θ such that $\sum_i \max(s_i - \theta, 0) = \tau$

1: $u \leftarrow \text{sort}(s)$ in descending order

2: $\text{cssv} \leftarrow$ cumulative sum of u

3: **for** $j = 1$ to n **do**

4: $t_j \leftarrow \frac{1}{j}(\text{cssv}[j] - \tau)$

5: **if** $u[j] > t_j$ **then**

6: $\rho \leftarrow j$

7: **end if**

8: **end for**

9: $\theta \leftarrow \frac{1}{\rho}(\sum_{i=1}^{\rho} u_i - \tau)$

10: **return** θ

2. Jester Dataset: A dataset containing users as rows, and jokes as columns. Entries correspond to a user's rating of the joke. This is a bigger, more sparse dataset with dimensions of 50692×150 and 78% of entries missing.
3. Amazon Products: This dataset contains 827 products and numerical reviews assigned to them by 622 users. This dataset is very sparse as only 0.6 % of the entries are non-empty. The choice is justified by the need to estimate the algorithm on really sparse data.

The experiment procedure was the same for the three algorithms:

1. Normalise the data
2. Split the data into training and test sets with most of the data being in the training set.
3. Tune hyperparameters by running the algorithm on the training set, computing performance metrics on the validation test, and repeating until satisfactory performance is achieved
4. Perform final testing by running the algorithm with found hyperparameters on the test set, normalise data back to original scale, compute metrics, visualise and report results

Results by dataset:

4.1 Lens Movies

For this dataset, which is the least sparse and smallest, we obtained satisfactory results for the three algorithms, shown in table 1, and we can see their quick convergence in the plots of figure 1.

Table 1: Evaluation metrics for Lens Movies dataset

Algorithm	MAE	Exact Accuracy	Round Accuracy
Franke-Wolfe	0.58	0.027	0.52
Pairwise Franke-Wolfe	0.60	0.26	0.51
Projected Gradient Method	0.59	0.25	0.52

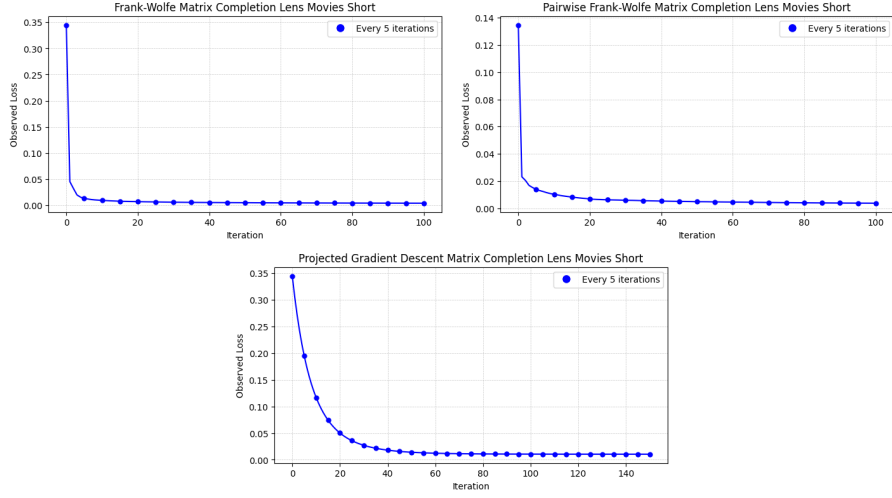


Figure 1: Convergence Rate for the different algorithms for Lens Movies dataset

4.2 Jester Dataset

For this dataset, which is much larger and more sparse, we also got satisfactory results. We didn't compute classification metrics for this dataset, as original scores were continuous. The table of metrics can be seen in table 2, and the rates of convergence, which for this dataset can be seen to be slower, are shown in figure 2.

Table 2: Evaluation Metrics for Jester dataset

Algorithm	MAE
Franke-Wolfe	0.87
Pairwise Franke-Wolfe	0.87
Projected Gradient Descent	0.36

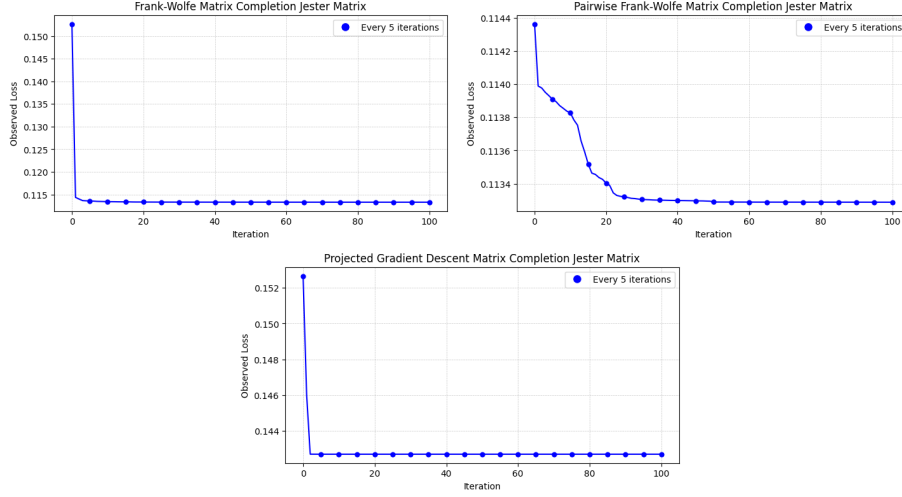


Figure 2: Convergence Rate for the different algorithms for Jester dataset

4.3 Amazon Products

For this dataset, our results were quite poor, but this was expected since it is the most sparse of the three we used for numerical experiments. On table 3 we can find the metrics and on figure 3 the rates of convergence for each algorithm.

Table 3: Evaluation Metrics for Amazon Products dataset

Algorithm	MAE	Exact Accuracy
Franke-Wolfe	1.79	0.44
Pairwise Franke-Wolfe	1.62	0.48
Projected Gradient Descent	1.95	0.24

5 Conclusions

The performance of the PGD method fluctuated depending on the dataset. On the first dataset it was evident that this method required more iterations to reach the same loss provided by FW and PFW; however, this behavior was expected due to the projection step in the algorithm, which none of the FW methods require. In terms of metrics, such as MAE and accuracies, all algorithms provided similar results. On the other hand, PGD reduced considerably more the MAE for the second dataset, which was sparser than the first one, in comparison with FW's algorithms. This was a very unexpected result, since it can be observed as well in the graphs that the other algorithms reached lower values of the loss function. Finally, in the last dataset the worst metrics were

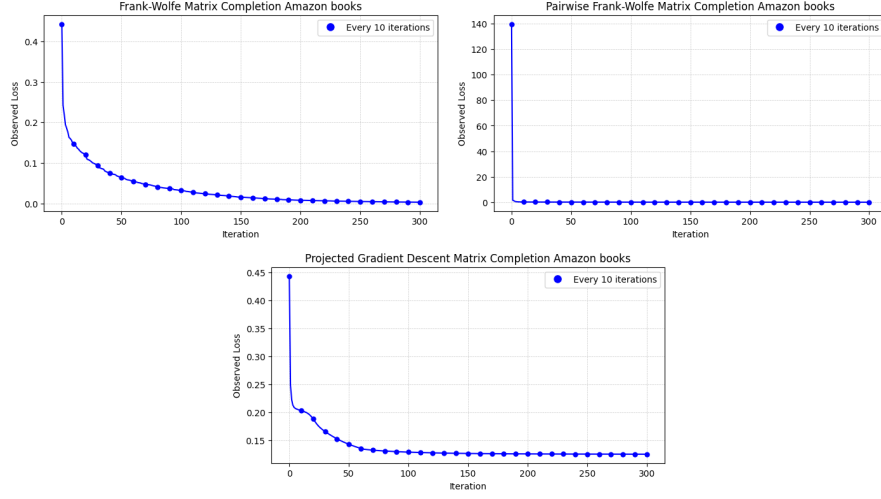


Figure 3: Convergence Rate for the different algorithms for Amazon Products dataset

obtained, as expected, due to high level of sparsity of the matrix, with only 0.6 % of the total entries. In terms of convergence, PGD and FW behaved similarly, but the algorithm's without projection showed better results in terms of accuracies and MAE.

From the previous results, we can conclude that the projection step can slow down the convergence of the model, and not always guarantees the best results. For this reason, FW and PFW are great alternatives of projection free algorithms. They showed good metrics, similar than the ones provided by PGD, even better in some cases, and a considerably fast convergence.